GRUPO FINANCIERO BANORTE

MANUAL DE USUARIO

Clave del Documento

Procedimiento para Cifrado y Decifrado simétrico / asimétrico. Ejemplo de componente en Java

Responsables de la Aplicación

Dominio	Subdominio
Adquirente	Medios de Pago

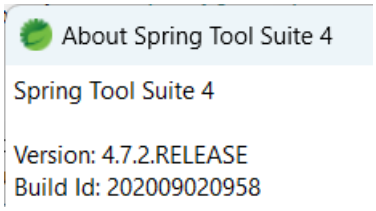
Control de Versiones del Documento

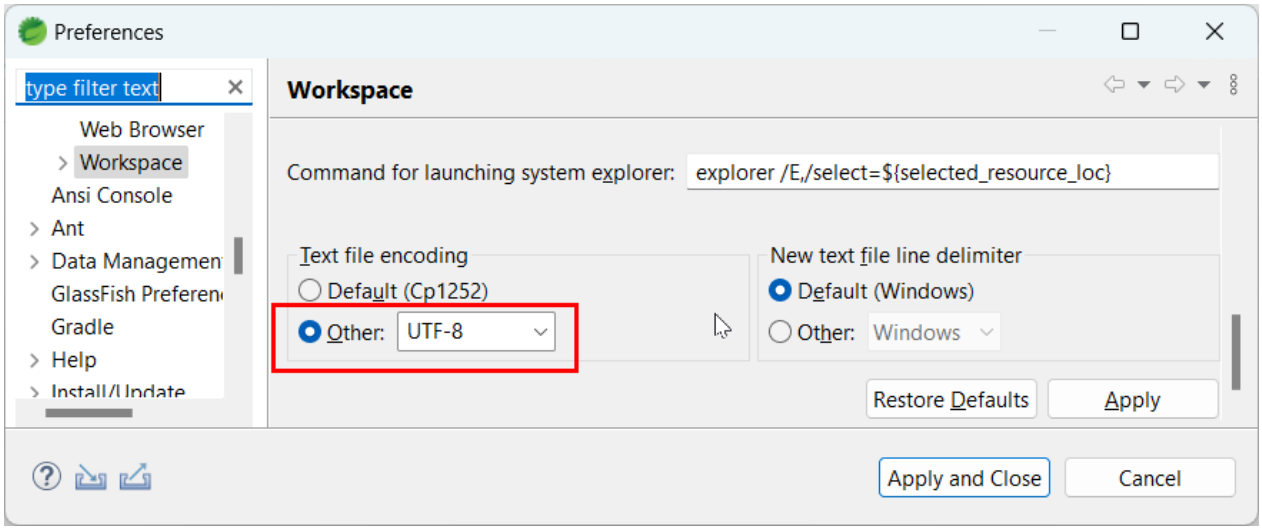
Fecha Modificación	Versión	Autor

Introducción

El presente documento tiene la finalidad de proporcionar al usuario del comercio el conocimiento necesario para poder cifrar y descifrar la información (mediante los Algoritmos provistos por Banorte) que se procesa en el aplicativo VCE (Ventana de Comercios).

El presente documento detalla un ejemplo de implementación con el lenguaje de programación Java en Spring Tools Suite 4.7.2.RELEASE.





Lenguaje de desarrollo.	Java 1.8
Framework.	SpringBoot 2.7.1
Herramienta de desarrollo.	Spring Tools Suite 4.7.2.RELEASE
Tipo de proyecto	Maven
Referencias adicionales.	spring-boot-starter spring-boot-starter spring-boot-starter-log4j spring-boot-starter-tomcat spring-boot-starter-logging commons-codec

Objetivo.

Obtener cadena de datos para invocar a VCE, se requiere armar una cadena compuesta por dos subcadenas separadas por el delimitador ":::".

Subcadena1:::Subcadena2

Procedimiento para generar Subcadena1:

Significado Subcadena1.

- Cadena cifrada de forma simétrica con RSA utilizando el certificado proporcionado por Banorte.
- El contenido de la cadena son tres elementos generados al momento de cifrar **Subacdena2** separados por el delimitador "::"
  - **ivHex::saltHex::passprhase**
- Algoritmo utilizado: RSA/ECB/OAEPWITHSHA-256ANDMGF1PADDING
- Del certificado proporcionado por Banorte se tiene que extraer la llave pública del certificado convirtiendo el certificado de PEM a x.509.

Código:

```java
package com.cifrado.utils;

import java.io.ByteArrayInputStream;
import java.io.InputStream;
import java.nio.charset.StandardCharsets;
import java.security.InvalidKeyException;
import java.security.KeyFactory;
import java.security.NoSuchAlgorithmException;
import java.security.NoSuchProviderException;
import java.security.PublicKey;
import java.security.cert.CertificateFactory;
import java.security.cert.X509Certificate;
import java.security.spec.X509EncodedKeySpec;

import javax.crypto.BadPaddingException;
import javax.crypto.Cipher;
import javax.crypto.IllegalBlockSizeException;
import javax.crypto.NoSuchPaddingException;
import org.apache.commons.codec.binary.Base64;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class CifradoRSA
{
  private static Logger logger = LoggerFactory.getLogger(com.cifrado.utils.CifradoRSA.class);

  public static String encrypt(String plainText, String publicKey) {
    try {
      Cipher encryptCipher = Cipher.getInstance("RSA/ECB/OAEPWITHSHA-256ANDMGF1PADDING", "SunJCE");


      byte[] b64decoded = Base64.decodeBase64(publicKey);
      CertificateFactory certFactory = CertificateFactory.getInstance("X.509");
      InputStream in = new ByteArrayInputStream(b64decoded);
      X509Certificate certificate = (X509Certificate)certFactory.generateCertificate(in);

      encryptCipher.init(1, certificate.getPublicKey());

      byte[] encryptedData = encryptCipher.doFinal(plainText.getBytes(StandardCharsets.UTF_8));
      String encodeToString = Base64.encodeBase64String(encryptedData);

      logger.debug("RSA Encrypt Provider: [{}], [{}]", new Object[] { encryptCipher.getProvider().getName(), encryptCipher.getAlgorithm() });
      return encodeToString;
    } catch (InvalidKeyException e) {
      logger.error("Llave RSA invalida", e);
    } catch (NoSuchAlgorithmException e) {
      logger.error("No es posible utilizar el algoritmo especificado RSA", e);
    } catch (NoSuchProviderException e) {
      logger.error("No es posible Localizar el proveedor RSA especificado", e);
    } catch (NoSuchPaddingException e) {
      logger.error("No es posible Localizar el padding especificado", e);
    } catch (IllegalBlockSizeException e) {
      logger.error("Tamaño de bloque ilegal", e);
    } catch (BadPaddingException e) {
      logger.error("Padding incorrecto", e);
    } catch (Exception e) {
      e.printStackTrace();
    }
    return null;
  }
}
```

plainText = Cadena **ivHex::saltHex::passprhase**

Ej.

7434316130317A753068723272746569::3076356C346F7366387675337A316171::g-T=Kq81GSgxRhc%

rsaPublicKey = Cadena que contiene el certificado proporcionado por Banorte

Ej.

MIIGdDCCBVygAwIBAgITFwAAxu3koN5OHp90hgABAADG7TANBgkqhkiG9w0BAQsFADB5MQswCQ
YDVQQGEwJNWDEZMBcGCgmSJomT8ixkARkWCWdmYmFub3J0ZTEeMBwGA1UEChMVU2VndXJp
ZGFkIGluZm9ybWF0aWNhMS8wLQYDVQQDEyZHcnVwbyBGaW5hbmNpZXJvIEJhbm9ydGUgQ0EgR
W50ZXJwcmlzZTAeFw0yMjAxMjExNzU3NDFaFw0yMzAxMjExNzU3NDFaMIGsMQswCQYDVQQGEwJ
NWDELMAkGA1UECBMCREYxDzANBgNVBAcTBk1leGljbzErMCkGA1UEChMiR3J1cG8gRmluYW5ja
WVybyBCYW5vcnRlIFNBQiBkZSBDVjEkMCIGA1UECxMbU2VndXJpZGFkIGRlIGxhIEluZm9ybWFjaW9
uMSwwKgYDVQQDEyNtdWx0aWNvYnJvc2tleS5kZXYudW5peC5iYW5vcnRlLmNvbTCCASIwDQYJKo
ZIhvcNAQEBBQADggEPADCCAQoCggEBAOW0T9LRmHPdL0gHFLojhObdEPnHbiQ0D84O3v96lZWH
dW19g8ZC772rMCkExT47qANVhCTEgnFhz80XOiMUSypODPWVkfcpgts13Lfxq07ltMUTK69M6fcVXjR
GBKUs3Gcv0kVYcCgKf2+q2jxTQeYRUraAzu6umZY5W98lJzqryED4Du3pNwla6uj5+X1g0Uds8oXSj4Ml
PjPUHedQ0xa/3GES+vMVCYB1qDnEwhmZFnLnzjCN+VM7mNGDP4kbGEYY+4asetd1QT2SIxmXIpnG
XWNHYeqNZoN3XerM0zVDQ/vgWkyLocSoocIxl1Bpn6hp/ycO0sTzA97FYUk5eJECAwEAAaOCAr8wgg
K7MC4GA1UdEQQnMCWCI211bHRpY29icm9za2V5LmRldi51bml4LmJhbm9ydGUuY29tMB0GA1UdD
gQWBBQBtoMxQxbByCO1tuY3s9yEFxpQ2TAfBgNVHSMEGDAWgBTQBPbPr59ruX3bXiA0YK0zB6UM
0zCB7QYDVR0fBIHlMIHiMIHfoIHcoIHZhoHWbGRhcDovLy9DTj1HcnVwbyUyMEZpbmFuY2llcm8lMjBC
YW5vcnRlJTIwQ0ElMjBFbnRlcnByaXNlLENOPUNBLUJBTk9SVEUtUkEsQ049Q0RQLENOPVB1Ymxp
YyUyMEtleSUyMFNlcnZpY2VzLENOPVNlcnZpY2VzLENOPUNvbmZpZ3VyYXRpb24sREM9Z2ZiYW5v
cnRlP2NlcnRpZmljYXRlUmV2b2NhdGlvbkxpc3Q/YmFzZT9vYmplY3RDbGFzcz1jUkxExN0cmlidXRpb
25Qb2ludDCB2wYIKwYBBQUHAQEEgc4wgcswgcgGCCsGAQUFBzAChoG7bGRhcDovLy9DTj1HcnV
wbyUyMEZpbmFuY2llcm8lMjBCYW5vcnRlJTIwQ0ElMjBFbnRlcnByaXNlLENOPUFJQSxDTj1QdWJsaW
MlMjBLZXklMjBTZXJ2aWNlcyxDTj1TZXJ2aWNlcyxDTj1Db25maWd1cmF0aW9uLERDPWdmYmFub3J
0ZT9jQUNlcnRpZmljYXRlP2Jhc2U/b2JqZWN0Q2xhc3M9Y2VydGlmaWNhdGlvbkF1dGhvcml0eTALBg
NVHQ8EBAMCBaAwPAYJKwYBBAGCNxUHBC8wLQYlKwYBBAGCNxUIg+v5BoK1j0eNkROCp58Ggu
HddCKCxphfgtj9DwIBZAIBDzATBgNVHSUEDDAKBggrBgEFBQcDATAbBgkrBgEEAYI3FQoEDjAMMA
oGCCsGAQUFBwMBMA0GCSqGSIb3DQEBCwUAA4IBAQBi1NI4gHDbgldA62xFDtt/9nsSMtPZ3yCi2E
vX+F8OnhecGDvq2i3/87gfTr9K0nWzA/3EwM21i6/JlcIyAJaKQQwu551Ic7ky5OBUw811be0MltwzvsdOA
+Nd641+Fh+VfW5/yaIQccXF2Zm7MBrCoXbDgzoX+leeIeqq55orBUqdA8/woYC796+ERHdQqpuVuowit
gnOZDFwr0Dm2kg02ZYHwUp5Pf55MfVYrT1AgVkP7X4KH1EcjNCRyYnOuVbUSzVtZSBYLCkhgB9Baf
YSvd78M1yS8Uc2y1wKh4D2TU/UpT3WXjbpA9P8Cn39WxX3EUfhrOwcUOtmkHA45Hcf

Resultado (el resultado es la **Subcadena1)**:

Ej.

X/GqsF19cIXMxzqlLCQWO/ZySIlL16JwJNiKSmS5pF8YjUbdK6a9PFpDTtafxdmKVyQG6unYSRCy0Gim
lyD+1jFvfgfYYF14Z1DtZU4Dq5ZOVhGXnK4DXUJhHM2T1dv3EtwFSurVMKGaBi6wGEX1W6xS90PvFP
Yv9x5NDnvkIGUvZmfE1HkP6pMUTZoxGFD1caLW13eSrsMkZ6JfeVo6kDUudKdiOMWrXFuHf7CT9Sw
9krq2Hwm8uUodIKMbGsXB5XOBZZqDAxeOsTaO78+DxgRnHtB+c2J3e0PuyJmZR0/PNs7p63gmUh44
yxlIJwFydQULCUAqGrPaONgTIgiHGA==

Procedimiento para generar Subcadena2:

Significado Subcadena2:

- Cadena cifrada de forma asimétrica con AES utilizando llaves generadas de manera dinámica.
- El contenido son los datos en formato JSON requeridos por VCE.
- Algoritmo utilizado: AES/CTR/NoPadding
- Se utilizan 4 elementos para lograr el cifrado:
  - **passPrhase**
    - Cadena de 16 caracteres.
    - Se generan 16 caracteres de manera aleatoria de una cadena predefinida:
      - 0123456789ABCDEFGHIJKLMNOPQRSTUVWXTZabcdefghiklmnopqrstuvwxyz*&-%/!?*+=()
  - **ivHex**
    - Cadena de 32 caracteres
    - Se generan 16 caracteres de manera aleatoria de una cadena predefinida:
      - 0123456789abcdefghiklmnopqrstuvwxyz
    - De los 16 caracteres se obtiene su representación en hexadecimal de cada caracter para obtener la cadena final de 32.
  - **saltHex**
    - Cadena de 32 caracteres
    - Se generan 16 caracteres de manera aleatoria de una cadena predefinida:
      0123456789abcdefghiklmnopqrstuvwxyz
    - De los 16 caracteres se obtiene su representación en hexadecimal de cada caracter para obtener la cadena final de 32.
  - **key**
    - Llave generada mediante el método Rfc2898DeriveBytes utilizando el algoritmo HashAlgorithmName.SHA1 y los elementos **saltHex** (los 16 caracteres convertidos en un arreglo de bytes) y **passprhase**.
    - De la llave generada se obtienen los primeros 16 bytes.

Código:

Generación datos aleatorios:

```
CifradoAES cifradoaes = CifradoAES.getInstance();
String ivHex = CifradoAES.createInitializationVector();
String saltHex = CifradoAES.createSalt();
String passPhrase = CifradoAES.createRandomKey();
```

```
public static com.cifrado.utils.CifradoAES getInstance() {
  if (instance == null) {
    instance = new com.cifrado.utils.CifradoAES(128, 1000);
  }
  return instance;
}

private CifradoAES(int keySize, int iterationCount) {
  this.keySize = keySize;
  this.iterationCount = iterationCount;
  try {
    this.cipher = Cipher.getInstance("AES/CTR/NoPadding");
  }
  catch (NoSuchAlgorithmException|javax.crypto.NoSuchPaddingException e) {
    throw fail(e);
  }
}


public static String createInitializationVector() {
  String cadenaRandom = "0123456789abcdefghiklmnopqrstuvwxyz";
  StringBuilder iv = new StringBuilder();
  Random rnd = new Random();
  while (iv.length() < 16) {
    int index = (int)(rnd.nextFloat() * cadenaRandom.length());
    iv.append(cadenaRandom.charAt(index));
  }
  char[] ivString = Hex.encodeHex(iv.toString().getBytes(StandardCharsets.UTF_8));
  return String.valueOf(ivString);
}


public static String createSalt() {
  String ramdom = "0123456789abcdefghiklmnopqrstuvwxyz";
  StringBuilder salt = new StringBuilder();
  Random rnd = new Random();
  while (salt.length() < 16) {
    int index = (int)(rnd.nextFloat() * ramdom.length());
    salt.append(ramdom.charAt(index));
  }
  char[] saltString = Hex.encodeHex(salt.toString().getBytes(StandardCharsets.UTF_8));
  return String.valueOf(saltString);
}
```

org.apache.commons.codec.binary.Hex

Converts hexadecimal Strings. The charset used for certain operation can be set, the default is set in `DEFAULT_CHARSET_NAME` This class is thread-safe.

Since:
1.1

Version:
$Id: Hex.java 1619948 2014-08-22 22:53:55Z ggregory $

```
public static String createRandomKey() {
  String ramdomkey = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXTZabcdefghiklmnopqrstuvwxyz*&-%/!?*+=()";
  StringBuilder keyRnd = new StringBuilder();
  Random rnd = new Random();
  while (keyRnd.length() < 16) {
    int index = (int)(rnd.nextFloat() * ramdomkey.length());
    keyRnd.append(ramdomkey.charAt(index));
  }
  return keyRnd.toString();
}
```

Invocación al método de cifrado:

```
String cadenaCifradaAES = cifradoaes.encrypt(saltHex, ivHex, passPhrase, cadenaACifrarDecodificada);
```

```
public String encrypt(String salt, String iv, String passphrase, String plaintext) {
  SecretKey key = generateKey(salt, passphrase);
  byte[] encrypted = doFinal(1, key, iv, plaintext.getBytes(StandardCharsets.UTF_8));
  return base64(encrypted);
}

private byte[] doFinal(int encryptMode, SecretKey key, String iv, byte[] bytes) {
  try {
    this.cipher.init(encryptMode, key, new IvParameterSpec(hex(iv)));
    return this.cipher.doFinal(bytes);
  }
  catch (InvalidKeyException|java.security.InvalidAlgorithmParameterException|javax.crypto.IllegalBlockSizeException|javax.crypto.BadPaddingException e) {
    return null;
  }
}

private SecretKey generateKey(String salt, String passphrase) {
  try {
    SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1");
    KeySpec spec = new PBEKeySpec(passphrase.toCharArray(), hex(salt), this.iterationCount, this.keySize);
    SecretKey key = new SecretKeySpec(factory.generateSecret(spec).getEncoded(), "AES");

    return key;
  }
  catch (NoSuchAlgorithmException|java.security.spec.InvalidKeySpecException e) {
    return null;
  }
}
```

cadenaACifrarDecodificada = Cadena texto plano que contiene el JSON de los datos requeridos por VCE.

Ej.

{"merchantId":"**********","name":"**********","password":"********","mode":"AUT","controlNumber":"PBA22042709695XXFA36H01","terminalId":"12348","amount":"1.00","merchantName":"Pruebas Banorte","merchantCity":"Monterrey","lang":"ES","customerRef1":"22042709695XXFA36H01","customerRef2":"","customerRef3":"","customerRef4":"","customerRef5":"","emailCdmx":"correo@correo.group","phoneCdmx":"5566778899","conceptoCdmx":"|Concepto:|Impuestos sobre Tenencia de Veh?culos y derechos|Referencia:|P31BFM|Ejercicio Fiscal:|2021","billToFirstName":"Nombre Cliente","billToLastName":"Apellidos Cliente","billToStreet":"Eugenia","billToStreetNumber":"197","billToStreetNumbe2":"","billToStreet2Col":"Narvarte","billToStreet2Del":"Benito Juarez","billToCity":"CDMX","billToState":"DF","billToCountry":"MX","billToPhoneNumber":"5566778899","billToPostalCode":"03999","billToEmail":"review@banorte.com","billToCustomerId":"","billToCustomerPassword":"","billToDateOfBirth":"","billToHostname":"LSTK","billToHttpBrowserEmail":"","billToIpAddress":"127.0.0.1","comments":"Compra a MSI","shipToFirstName":"","shipToLastName":"","shipToStreetNumber":"","shipToStreetNumber2":"","shipToStreet2Col":"","shipToStreet2Del":"","shipToCity":"","shipToState":"","shipToCountry":"","shipToPostalCode":"","shipToPhoneNumber":"","merchantDefinedDataField3":"","merchantDefinedDataField4":"","merchantDefinedDataField5":"","merchantDefinedDataField8":"","merchantDefinedDataField6":"","merchantDefinedDataField7":"","merchantDefinedDataField9":"","merchantDefinedDataField10":"","merchantDefinedDataField11":"","merchantDefinedDataField12":"","merchantDefinedDataField13":"","merchantDefinedDataField14":"","merchantDefinedDataField15":"","merchantDefinedDataField16":"","merchantDefinedDataField17":"","merchantDefinedDataField18":"","merchantDefinedDataField19":"","merchantDefinedDataField20":"","merchantDefinedDataField21":"","merchantDefinedDataField22":"","merchantDefinedDataField23":"","merchantDefinedDataField24":"","merchantDefinedDataField25":"","merchantDefinedDataField26":"","merchantDefinedDataField27":"","merchantDefinedDataField28":"","merchantDefinedDataField29":"","merchantDefinedDataField30":"","merchantDefinedDataField31":"","merchantDefinedDataField32":"","merchantDefinedDataField33":"","merchantDefinedDataField34":"","merchantDefinedDataField35":"","merchantDefinedDataField36":"","merchantDefinedDataField37":"","merchantDefinedDataField38":"","merchantDefinedDataField39":"","merchantDefinedDataField40":"","merchantDefinedDataField41":"","merchantDefinedDataField42":"","merchantDefinedDataField43":"","merchantDefinedDataField44":"","merchantDefinedDataField45":"","merchantDefinedDataField46":"","merchantDefinedDataField47":"","merchantDefinedDataField48":"","merchantDefinedDataField49":"","merchantDefinedDataField50":"","merchantDefinedDataField51":"","merchantDefinedDataField52":"","merchantDefinedDataField53":"","merchantDefinedDataField54":"","merchantDefinedDataField55":"","merchantDefinedDataField56":"","merchantDefinedDataField57":"","merchantDefinedDataField58":"","merchantDefinedDataField59":"","merchantDefinedDataField60":"","merchantDefinedDataField61":"","merchantDefinedDataField62":"","merchantDefinedDataField63":"","merchantDefinedDataField64":"","merchantDefinedDataField65":"","merchantDefinedDataField66":"","merchantDefinedDataField67":"","merchantDefinedDataField68":"","merchantDefinedDataField69":"","merchantDefinedDataField70":"","merchantDefinedDataField71":"","merchantDefinedDataField72":"","merchantDefinedDataField73":"","merchantDefinedDataField74":"","merchantDefinedDataField75":"","merchantDefinedDataField76":"","merchantDefinedDataField77":"","merchantDefinedDataField78":"","merchantDefinedDataField79":"","merchantDefinedDataField80":"","merchantDefinedDataField81":"","merchantDefinedDataField82":"","merchantDefinedDataField83":"","merchantDefinedDataField84":"","merchantDefinedDataField85":"","merchantDefinedDataField86":"","merchantDefinedDataField87":"","merchantDefinedDataField88":"","merchantDefinedDataField89":"","merchantDefinedDataField90":"","merchantDefinedDataField91":"","merchantDefinedDataField92":"","merchantDefinedDataField93":"","merchantDefinedDataField94":"","merchantDefinedDataField95":"","merchantDefinedDataField96":"","merchantDefinedDataField97":"","merchantDefinedDataField98":"","merchantDefinedDataField99":"","merchantDefinedDataField100":""}

Ej con formato JSON:

{
"merchantId": "**********",
"name": "**********",
"password": "********",
"mode": "AUT",
"controlNumber": "PBA22042709695XXFA36H01",
"terminalId": "12348",
"amount": "1.00",
"merchantName": "Pruebas Banorte",
"merchantCity": "Monterrey",
"lang": "ES",
"customerRef1": "22042709695XXFA36H01",
"customerRef2": "",
"customerRef3": "",
"customerRef4": "",
"customerRef5": "",
"emailCdmx": "correo@correo.group",
"phoneCdmx": "5566778899",
"conceptoCdmx": "|Concepto:|Impuestos sobre Tenencia de Veh?culos y derechos|Referencia:|P31BFM|Ejercicio Fiscal:|2021",
"billToFirstName": "Nombre Cliente",
"billToLastName": "Apellidos Cliente",
"billToStreet": "Eugenia",
"billToStreetNumber": "197",
"billToStreetNumbe2": "",
"billToStreet2Col": "Narvarte",
"billToStreet2Del": "Benito Juarez",
"billToCity": "CDMX",
"billToState": "DF",
"billToCountry": "MX",
"billToPhoneNumber": "5566778899",
"billToPostalCode": "03999",
"billToEmail": "review@banorte.com",
"billToCustomerId": "",
"billToCustomerPassword": "",
"billToDateOfBirth": "",
"billToHostname": "LSTK",
"billToHttpBrowserEmail": "",
"billToIpAddress": "127.0.0.1",
"comments": "Compra a MSI",
"shipToFirstName": "",
"shipToLastName": "",
"shipToStreetNumber": "",
"shipToStreetNumber2": "",
"shipToStreet2Col": "",
"shipToStreet2Del": "",
"shipToCity": "",
"shipToState": "",
"shipToCountry": "",
"shipToPostalCode": "",
"shipToPhoneNumber": "",
"merchantDefinedDataField3": "",

"merchantDefinedDataField4": "",
"merchantDefinedDataField5": "",
"merchantDefinedDataField8": "",
"merchantDefinedDataField6": "",
"merchantDefinedDataField7": "",
"merchantDefinedDataField9": "",
"merchantDefinedDataField10": "",
"merchantDefinedDataField11": "",
"merchantDefinedDataField12": "",
"merchantDefinedDataField13": "",
"merchantDefinedDataField14": "",
"merchantDefinedDataField15": "",
"merchantDefinedDataField16": "",
"merchantDefinedDataField17": "",
"merchantDefinedDataField18": "",
"merchantDefinedDataField19": "",
"merchantDefinedDataField20": "",
"merchantDefinedDataField21": "",
"merchantDefinedDataField22": "",
"merchantDefinedDataField23": "",
"merchantDefinedDataField24": "",
"merchantDefinedDataField25": "",
"merchantDefinedDataField26": "",
"merchantDefinedDataField27": "",
"merchantDefinedDataField28": "",
"merchantDefinedDataField29": "",
"merchantDefinedDataField30": "",
"merchantDefinedDataField31": "",
"merchantDefinedDataField32": "",
"merchantDefinedDataField33": "",
"merchantDefinedDataField34": "",
"merchantDefinedDataField35": "",
"merchantDefinedDataField36": "",
"merchantDefinedDataField37": "",
"merchantDefinedDataField38": "",
"merchantDefinedDataField39": "",
"merchantDefinedDataField40": "",
"merchantDefinedDataField41": "",
"merchantDefinedDataField42": "",
"merchantDefinedDataField43": "",
"merchantDefinedDataField44": "",
"merchantDefinedDataField45": "",
"merchantDefinedDataField46": "",
"merchantDefinedDataField47": "",
"merchantDefinedDataField48": "",
"merchantDefinedDataField49": "",
"merchantDefinedDataField50": "",
"merchantDefinedDataField51": "",
"merchantDefinedDataField52": "",
"merchantDefinedDataField53": "",
"merchantDefinedDataField54": "",
"merchantDefinedDataField55": "",
"merchantDefinedDataField56": "",

```
    "merchantDefinedDataField57": "",
    "merchantDefinedDataField58": "",
    "merchantDefinedDataField59": "",
    "merchantDefinedDataField60": "",
    "merchantDefinedDataField61": "",
    "merchantDefinedDataField62": "",
    "merchantDefinedDataField63": "",
    "merchantDefinedDataField64": "",
    "merchantDefinedDataField65": "",
    "merchantDefinedDataField66": "",
    "merchantDefinedDataField67": "",
    "merchantDefinedDataField68": "",
    "merchantDefinedDataField69": "",
    "merchantDefinedDataField70": "",
    "merchantDefinedDataField71": "",
    "merchantDefinedDataField72": "",
    "merchantDefinedDataField73": "",
    "merchantDefinedDataField74": "",
    "merchantDefinedDataField75": "",
    "merchantDefinedDataField76": "",
    "merchantDefinedDataField77": "",
    "merchantDefinedDataField78": "",
    "merchantDefinedDataField79": "",
    "merchantDefinedDataField80": "",
    "merchantDefinedDataField81": "",
    "merchantDefinedDataField82": "",
    "merchantDefinedDataField83": "",
    "merchantDefinedDataField84": "",
    "merchantDefinedDataField85": "",
    "merchantDefinedDataField86": "",
    "merchantDefinedDataField87": "",
    "merchantDefinedDataField88": "",
    "merchantDefinedDataField89": "",
    "merchantDefinedDataField90": "",
    "merchantDefinedDataField91": "",
    "merchantDefinedDataField92": "",
    "merchantDefinedDataField93": "",
    "merchantDefinedDataField94": "",
    "merchantDefinedDataField95": "",
    "merchantDefinedDataField96": "",
    "merchantDefinedDataField97": "",
    "merchantDefinedDataField98": "",
    "merchantDefinedDataField99": "",
    "merchantDefinedDataField100": ""
}
```


Resultado:

XxsmFcAcjfmJmbaX0wbHDB0a/+lId/gR3kkYx+dTve29uvwaljrgcbWBfLShTNg6Scv2Dr9MQMot831PQUUGfopP
vslJDws7D75+tLA13EW6UlRb1aRYhSwfJ3MCLqlSNyfDMHSDucG0XvWvNwT97S1MMSmpCiQSEeFaCufnnp2
Y3cHhuNcoxDvrwdA5IbcOBjuuUqOYCTxEgADOKgrwTkDRVtzJ4psGzBI8I9i0MKYUVrMX8lVxvQLZioVrPYiBq5
CnprnTIdYIw61z16tFxVWA328cZNuDrRqM5WX2XEv9bHhjqC4gHImJ9B905ATzoeW79sy3OCtofd6ZGrv/W1Gb

eJyOoanc3bhiKWv49Y/ojB8xHJRw3mKX8HLubnK49HF10eacGbHhM9sR8eFFcZ13mJhJ6vJWlNR1geShxj9tcB
bKvwjrCenfchnudRiYu/HYu90szpBitINcPl0vf3jzYcHLqFGWmGcm09udDLyEE1pm0JCtLMh9HNFtUcsv2TGmrZ
WiAa1vGsVBOAtB0P04Yk6+fxKPQyO6xEcSLDDshR1rAzuXrD+1QW4XP61x/Anje830guKIVm1Wv3zYy8AZ/Wj
QLFfXQSMgP8aoglXWMl+aZkJVZ+bm2XJEoWuJuVT0rRl1qM8sQyDvIX014StmTcPT5ppKFiOSi6gBOYgvu2P
X9vq8WYMLFGqENS2RY/qJGmJEOAew3wHyTnU9CcItdbjamyr9jktSpYxEMuxGO7G+z92RMdvKe5FnTKHljeI
B8x10fx7rO3x3U/XO8J8MzY+U3im7yccfspKF+wPDiKwaRTQ/7VNYpNiSSAeGOy+tMcwVWYHGCph5U2MBfU
Se5yX58NruIyLnbAs/3IeKWvuoP8WG4FaYN8PNb3lVoxOP9+b1d4jr9yUyRKDbFRW74K/OTdk606AcHph5juQS
bQRW0yFAb9IyskIBCUU/xUR840L6I18YgB5iBSoKGgBaGGo+RM6g+9P8vElT1Zy/LxB+2SvGU9Vz7s753TI2d
C8cJf3uO8fxoC5/VKunJDNhXkjW5VeJXqTKyCei42iQ0P+lzq0N5bVFXnM0EFQm/DFt4LlQVJHtooMSZ/xAP0gO
j870diVF4zNJxeICCyFIihApjJZLMlRGnDBqyMmNSwWoag78qs6EcohjN+D6k82RjkHAgA6EPUJccIP/XRQKerT
yO+NOdO5M1Bv3IYHBj9gHHW0+JT1pYFyp6c9tqtwgHuGpFKRtJcQF9mcz0mjfCPNh3V79IGuIEbhloYwckFJXb
UJQ3O0rGnVWz/JIOR7FL2Uc5v+IccsHQ/N1RDB+XRj5wpBJN4cJzy05xa6tPRjZXrWNboVbaa0142iq3mqFBND
A9fHHqAV8frRPQ6WG22lbnYjNT6tB90T1+fak7XoeWzZbyQ71X1Djqdz86pIi1rfEjmp+aX2fts11HQoBKdeBq8nF
LCnsQ2/LceymoSNNBmyMPjR3cP4j976uXhA+nAIMJbvaT9tJHLpflhP6R8ex0e7roPdbMkkLqKfal98/iahvNW1u1
zNQoZHALFOwdEtuMWXErPZRZWC8zc+Bdt/wL60kaWx0eqpbK9gL1dPT+C+e4dwcFoITSIEqGsx0XqplkAsPa
1FvTZxCn2SpLIaFjMFhg51aNaJbmI+FYcfvJ5EcW9UDs2s3z12PLjY0rpPCsAq8iG2vInlBg1i42FbBi0/Pub2J5GU
Z3im71xDvW4dHlCZASep//hNtIARvTPnJQDrVBJQ1mZvbiUZw1bUcqO9tt414LN5CQzPi+8SY0YrabgltslwHqjcT
ImDT4/L5jueSlW1FmFW/iX1Q4PAP0DNywdhMAwDxQUgMHBRslfwoqmG6YOg4w2QnVuiCay7UwEeScK2b4D
q0pWzfOVmBwWBLSF34NRcDGfluOhHdX9i3p7uFy1acfcaOaW26g1vNo+rTROcujqNHnE9CGRjwT9ePwDsm8
bwdPvjHaMdgr0pacQLkV/IXLZjjM5xjr5GHs4URkMar12OF5CTRi31CfAH2v4fPtSuvo6jN7xaMkysjk44nzUWQzre
o4HHFS3dbbRUBcYWtoH8slEOqhrGKMYtTQjoDg5vb2lJVoSaS92AI9XOYP9UGOpPD8idHC+8cJ6S4cKzI9ISg
EREyG3tdTttJNnw4nk3Q8D0ciC/78d6sAaEK4dtVbKLdeBIyoSORhLuE+1yEY0YtU/WZB7wdKJHo72Ht1JM44Y
hr1u3mvR7xDjKhbeoD3N5E55yyMjI+9sqi7drfjpgc2MJh86u/70bgRpTAMG/h2zXFwBymRONjTHr26uDq/wLIYvSt
XGJg3DyYtQgL7PUIAPbPnWE/+4zq1LTWW/whYh/g7dVGZtRnTSrBY4+IekkSi772EKvPudAOVkDN3LIe6nQzw
HiQgvLhUKx2eFIX9Q9gYN23QqSPuTWCgcSudSST/FY4QEddRXOnUxi/26OUAawMieS41MELKblGUSpQhfBv
lPDCxZ2YlEjDtc4gz4Fd6zY8Ch0XL+c6baSVALRR24PLDSMqr4Ea/yXyCQSFVpHNg5/K3BTQZRSMok+h2RT2r
eiBeeUtVji00j6SWiGmQszF1IayFxvzfjG+6f01jpRYhwzsLhkmMyOaiSklAae8PRKj9xsQhGp+3a6ct195ln6WAITkb
YwQ/8zO+zgXRmxuq/+Ku29KfR5Ul7ZDAwejQEXsGEFw9epzfXqRvU/WoRQADvfOWO8WdK7I6vI2YKe/zWQO
WCqewDRV9OQ36UcdxHhY28RqSEMgcZFC0MU2HQ2qZhY7+oT77Xm9/grm//qUIg+fxe2UDrbEpohC0u7ZZ4l
F9sRfBya+bPjRx453KSeWvFjvGqi04XNlSOGwmP20cmLWyw1glTsDrHQ+D+9ugw6aQXsrvYR+H2wfJTw4S9s
Qa6t3ZRXoTFeMTdP6VlZA66EeV3Yn/QxuIbQ530CJulJM+ndTK7mv754mhRAuJr4TeWuWE8P86S+I/H2jiGxm
5RKWUTnfTNnAyfTn904nY+pVYNoE2TMjd2nJnXM2JwZMR0BKOou+HkSyBNz4La9LaAJKsQ+axoaOglSPMg
gy+kSRuUsMPQwdk57x6/ORUitbvy1pQ9wXVAyj5LeXZxmHYCPnajmTE4sLOVrexIPzW3zRDXrKDuKObQHSru
6t5hXIZbL5I4IvJNernXazLo333mzpBMgP0on6p2LZHdwcHQtGpBw+U0evMJ5qW5TVbYorWKeOYwEYj6OjH1D
98qr8c7Ywqh8+qrgh9gLe7ngNxIRd+q+jhlA1Sis5thpDwxlmx/E7cFW3mnVeGXElk12U7Ozq6bPJuyBXcHVQ0im
LgX423dQ6dQzzJwevmfEo/4LiX2C7xdcI0jdT9TnHNJm6YbdVxtK2Wy0WVcXcTgyLKKhyA1monxqSRnddPCVU
RoCWijLM6FJCjfkO0TgOBgLS+9mRXMd9BTDKqyttueLHbbLVJnupi/lYNZsuDYqipotz6Xw0yRTZA2hZCrQZtrD
YtGimCS1EF5Ljqf/pyXhk1nEEtuY1pGwsKc4wEZGNKI3yErhve6GyaFEQ0IXE+h9G5F8oPlMxpdQvHTCvBwv9y
zg7bvG6q/4HY/VZc4i4Y2hVPpB4kKqQbw5ULQCzLC7v1j8yrlEKUvriOkvOn328TrM9Oeag/kYFctT278nrL024a6
15Gq+NazJe6sgaC4HSwCk19uXCYEyI9Mu5ENeEh8YJI3YidWZk979zyumJexQNFiC70GsHRo5IURsmMjn1+w
ofp8B+hIzRORLOtANeGhb2v1xy/bvqN5d7eTSdpsiwZFcWHcM1ZdhncdCqYQVzDlhFdEpeT5hrVMzmy/a17/rgR
9E2G3s2wfMGCeYxTMgZIsaOYyJVTOQTiAh2F59NfBqiVzFxAvUW5KAtUmK+hWBPH1HAwH3WffuZ4ACuUt+
Q7wLJUctMdtVO8vWPSr0XtduZvsVAGE48lxrEwwyG4dJdCsBDqyhDYOwywLy5Nvnq6eD7FatnLK0ZhMskr3sk
V5pfc/P5++fUAiFoTkl9b/6Eifk8PJCnpEaPP7hZT3zDLNdx9Eb9wxYgE09ulslKaPchCE9LGUSOm88V/0uECL3gt
nyJaL53fdl/9V0MyOapFkyfq99r1I6J3UtzhTVIxJ4j72t8EwE5OancyHdGuknxF2SO5u0Y5In4uPDu5Q/2xPUeo1T0
n3NA25VFPKWJ077/Ti4+YUksXMeo3u/pxGezB8Fr6swCvAwrpSW+qUM7+O0vBxZd/NEfjcxs3H9PjLqY21rsZ+
HMc521iPLq15rJhX+dkbKqhFkwwGKdn6Xonh5NweICIx9smeeBc7J9aq1E7K+Nl8gbIOZ5esnqyWAM9/QxG10tf
yrJvn+n1sHtHyzExohgYwM+DUu7FOllFEmqpFLP4ex92z3sEghR+dJaYNHpPBOsE2vXkZY4JW45GJONFqD8k
P/k5PGEcHH3VrOS7+yeRM0WZps9SM1PMEyEHEnIbDR+e/sUf59/wL2/tRmDSgp1AsZ8ILJS5XxTYV8YbZoOg
TyvvrE+pc0izN0b5h2nTdQBXFRr8r8poAvXD1g29HyqChXYC9MZvWZmU6Dxl7Sz/0qjzRUx+szpQMZwzsrXRQ
DWtuQhC8QDM02T9F1mUccTkZ0xGb9CIa4mfAVsfjBwct+gNZujcGdKQbbm0RkBh7HvFgzkE/ZeLWghx4QbXF
w5NdIBYgxL26xvSQiPZXYXtFeEdG72hnHTHu0b9NWkHuv9jEDJqPWaayUbsRHmPC/8jaT3dsHMeNZA2MXX
AXLrfUuVdNpgPQAib4M7f1uM7uDrtFQnfAX7abrT6l+o4AI8v2w7RKrFPFymUgaJT5T89XH6jDTXdCIIMpQcvcPI

pfrY9SkMVyvqyGk1q2A3ebE1JGpgJnOPsiJ8anX4qWjjbhQahTV9b660XE4gaLDqdAY2BY2kjnIw7GYDGJhtHvX
lQk7nOmamFutfXXfgtKUylrK5OWE9Y97EsnMyc5pUTqeihpP07IY0R/OYQxuiIrxLs+JT+gxkm44yP+6alq1O3Ho
UGnb1gyuFfSdqAkJAidwSjZ5dsdVP7B4NeN7bGDIuN+Bd/06bgaa0dcd1nU5Pmlo7nr41U23eHiPiBFUcrMml5FI
dkm4CuWEG794hVXPcFYrI7MEGHyDZfN9rYt+01raza1w1bzG6G9xzRMqpVCdoug724fEKmqEaWi9vrcJxds6g
YUJsCBk6BwNSZVze4B3MTkVbGgnFqpzh1CIhCBTYCqvnc1lnwujGVRiLNqvR4mhTcNuD1nLG+AvTnrD9gyy
WQOJ+Lng4jMajhbGuhipQ4Z/kKCx2K0Ji2HdxG6Iy3EGhH9r0LMLE1W5OB9azHTsVrHdx3mTmcEbWWUd+uN
mWCzW1WcTIFfCQxUJaU8ltL6qBagPsPZii71Zw0h+AkHs+IgnZjKD3FMbUXNMok1z4/bSGla7Gf3SzpIko44eijx
YRM9lSnppvHTnpKHaafpoPIURE8r8DtEgzHM/Av7/95glgm/JWpigbUU2T5VEm9RRc+jZnuM0Yc9GnLa8NZda5
mKnWsyf+IQznyNfWdFjRM5PMoYNglE1yO1FG6yazUPZ6j9732LbGdoHyd9AThP3jrQZiYgIZOM6An3vZXyc01
RTEel3wabbBww1YXxAoMpl5SAhtuXsJj5CxT4Ihxrty3rup4NfUBzJ+Hn7Xjqy1QxmF2RLKJlLkPeplTLYS0TTqMa
8S4wmVZ4Lleqoj9C5L6N6adzrjauWmkflhq3RF5JYnBLPwgCoee4mX3jkx/LsKZpWE70SmN5MbWwsPaO27Gn
AH6WVvwiM125A2q253iI7iCRrguxOTbqBVJow6NnKOiHX26hRMjNaagEOrqC6okII=

Armado de cadena final

En base al objetivo, se tiene que armar una cadena final con las dos subcadenas.

Subcadena1:::Subcadena2

Tomando los resultados de los ejemplos la cadena final quedaría:

SGVw18pZAxZwtIvJEtmYjNqC/Gy3nD5ae+IqOXGBY8HNfBXhPx9u5bbQqr4hE+MhGxkga1XaA99KIB+g9msZh
BZ8PdSWTE6FcJhfu0c11fjz8b/90TFCJ6MXwNJaX6WmVKTnvpLhKGVrcVO+RkhuwHUA68jrNqvaQYAJtwvIMf
PDwp/YFFFkFLAS6NgPUc4RwGZmsPAo5Km44ImoTJzHo7Qwj/gCecRtIdVs8WrWiyLNY0RScf9CkyznFsf6Mv
QMstw1dzg2tAakXd+yoAVgKalr8XnkzMnxKL3z4EONWsD/J4eIoaqs91aAxCyYbC65I7bqyYNbaKBGp648pdhdz
w==:::o6VlkpaOn4DgWF+jwNssqjI4NtXCvSJ+jT/AW/evMa9m+9+FqXRLgrGJMA2PyASkJtZpIKkRRktibxw+1V
K+/5W4rdnYsGAodMyXRCgIGL77M5RBp7eO/7bvAV2smrUieJJiLb/e+Dc/MnXIVGKY8RprtibwfncwUOj6qE+wH
uf3teGQBuJuTEwrq7QnKCNlO2gYW9TiDhL3ruY3K7DpzsPL9CwU7bQeukH+BH8PMz3xqvjvhuLj1boBRb/UuV
qDGoqVRCammOljXoiUZEFXYZMLseQhWkTcngK5lh/GjyCUfjNz/VLnjEguxSDALKC2o8QSmQ/NkSAdi5Fv06kp
YtjK69nY3tQub/i+9Yc/YQ9i8cAbGpVu2sGjFoW40BAR3bBMRpp0z1+YgxAelVkym4Ai3N/slFSp8CtgE1S3TK0W
yNYmcOM61mAa5tyIzDsb+HlxNxqgmJj1qZm7tNzqs+tsVLJ/AigvSFS5PM2CIG23hYhFEGy9fuMNf9uI6aJttaF3a
9uJKEOAUglvxJanKR0S7YJIC/iv6K4+xH7PTRRe6WOrC1M4BS37lXiEOGixXUgldX1Ry7/yWJOGLYzon0pZFV
SIdcvnD/6DS+swvSGwRic+WFgGFC88yurFwW3O6WhZENxhrsg8CpvRbdjqRCFa+5iflLjd8IR1opRdPmAPk+sh
LvTdNtu1uRkffD9GwipVCQcoxEQLno+FT/fHSPiLj9N2fcH5scMkD2Wzot2HCCZko+o6lntrlxCrdRL9utmRYCOrkg
t99QxyyHNqNOLsbmf16rHIjGCvaJCndBe2cPPDR1/igwn3WCuhumdkEJpVd6gplHVU9W/FIzIQYuyzEBOO9G2
3Xk3d+UWg7mOETH4Gmj/8RcPRvuSBF1HSTuh7RDAo0eWtqnAl82e+J6dFZQYAX1mljqzoaxsfD6yQD4gtyG+
41M1qmfhvEp1tLCUV8m5Kh2F48c1MYy/7t47ksPeKAtEMnu5Pyn5nODoDKbrcHnDpi0EdNj1lnjltvkMZNtWmnoZ
3D/5UXmxw7xwTBgirfqi2UOda4a3OtGh6Aj3EJULKAYNNHmK9E0EY7FM2cfmUCMt3n83ZDYtwPDpg4HqjwVq
NProHHS485ez8GncKdabSTrO3a0iukUH8EkzdvhYvD4RXlapSdevPVHZYGuiHLawn3O82YmpWbdOe3/csk47J
U9PxlYiJ0FiV8mHhv6xxRl3JnsQHnAaNs4FOdX7D99ziCg9sOIde8b7ARkqPRttJ7OdtjR4ZDq1ZIQHBL1cttvnGgj
K+FPdLxgHCDQddUt6+3HGmaQdPNK0feqGBYYNqskoufoisR0rBn+Ib4SxM+k+JGBDP+oFkKV/N2DHAmBifyrf
AZx85plv+Bpmmhy1Bbt8z0aRchqMsv1pM7NwsB6mu/ZUV6nKXc+AoFR8+qKc/skhPnxcSfD1/YI+ZPvFM5ku8P
swDaR3Jsa02NJewtuvkTbD7OaYI9BH1VbbTLOUOa6kLJ/uwpOHoj525nldS7+OfUnMekql6Vd3nAyVIPdoVtPgr
Mg327knTbhXpRf0pmhR//h7ohlQ+L13Xge3oZwN51VEsZxLZkIWx87G7FmvmJVCQw+yQMAogwvK8xUian28U
TvHl7bFn3/ge2PpzE7jbVKI774hsFkIuMR65W2uZlgd40wawEcOygIzBr+QCmeFlV0N1jAPv+Brbq9bjHkGNdmEf
3TEt6F9fTreBcTxEBkjI507X1vjh69TO3AKmVIFjXBy1uQr88Wqf2mc2LDVO6h4pXKDq1ghsQtTbw85TB+hTxNe
XIDcwEvgy/j/+B2qpocBbiYtwjEtiYoVPRBsoQ3yUOkVAeDK4BRD10Q/VAhrEDmlVbdjEXeFFyq4+hUgItzZ0SthC
htXnRLidIscagLYAJzDdX3yPQWSM/SSGLxHo8n3U+ji6LZly0dMegHFktwulc5MiSN/4paBMUPfpyuNFitSUK2gZ
0NeFM98DGMsYETdmGRDOyYmVW41x2udKaKU8E20uN6HnHtep/UOhT0b5gtadx6ibsg7j6g+jQO3w2sJb3yN
LoUgYCJKd46M1bCV6u3Og68OaFjvdgIUn0FUm8+otGFiOVvv8DlM2eMAmH2OeCDbwO0zGTfgLvH+1Xxce0P
+3aQz5nkkCMyDGircBQXOz/lQZPg0HyTWDxUexcooxr1qOsZ/0HGN10X9nvWgLHBdmsIUWhqEncoBcy3yHS
HEhEIpLWE7iqrHmv00eP1/eo2aoMO5Fkc2U4l+QlGZHnb+ar6vxr6cAknMpRiHDcQXY8a0wjS5VW7hxf6Va9xiB
b6EG/71OUpNrxkdoUv6/w9J4or/zykJbKyCRvut7GPyx9XlASHmDV59IkZ93iwHNyy7aGNEED46ljATTO6Fwvw5
pwOEt6LaYFc2aUSl3wz9XF1fGjmBJuxKwVddjTBhssmmcyziZrHG2ZZPV9LQvvlte1aHoJhcT1tdvB2jZfvhl3Gban
4R1EyPYBpEVi/gdXbYjxs35mcm1hbGHyqyWKAQPgctktPC835e04beESX6V/xxlQv4p2v/IvVVXRe27UXoB6RQ
CGAwF2XiUUISKzAqYiLTUAqkHa+SoGcT4FWxufm6D5hk1B7FWLKUTDcHQbcBUiS7fsELsrbk8kRsZQkCLdku
6stxod+QQvLlfRqpQRJejHl+S8sSU38qNQO0gc8o2cBswVhrFSos07gU5Ibwi5RMSRJkVsi4nHqvOKs5G1V0HP
9nMklwgnFPCcgwgrGfAP08UHykiV3yWih8t+jN9sdsWl5lxZ7CKuqvn0LBWthf3PRprMpkiyUY/9L74D//FxQ4ejzO
y6mfCz6Jq0nZqHJk21CTB4bgW5Gp+/YcDGuco34OcAOTGeHGHU/olfwhKdwIaJLhnMEKlE07FbXl5MM8VnzrP
qe/RQ8FQi9IIOjQrtkAjTLYBKxlSUfQqS/dykHI8srCSxY947HX34Qp4uIr51+P4ghqfnmh9pbbXrasquTP6IP5lSO5n
H1qMHXss6wVm9Ah7KuWPgnY1pUWRYMBHcYILV3u/o9PBhyaeTRNw4OG0w6DzonWRW+Bym8cI61TpIQ3I
mrWsHF/bDphcwdChlI6KKUzlOb/6N/+ivQwEDGus5QnIzy8mhyxpgh76OGoWDocLHUzbCPSgzrT9vdRe9Ps9N
adMjct4ITP4RYfNIlk8S+4Al33bT8K86Z5WO0rBgvvd7bkAcan7YE5KCxZuzQQjaUZNeRPDTzZ29NVuKl6mabk7
rr+P6u1AiOMLgNHddLb2MSF06ymkDM0qkqDrTTzMoJMAAUrqYyitv3M3Y1mEE2WsVki1riRoSE0BuT0Cv7kb1
UTkmxlu/ftGh+4xs7+SCMGncf/pqjMvKtCnVpcwFYaOiexUrb3+s12Jvbk1UwTBIIVF+ytN7jXokr3bpBtxaYI5Pfht6B

dOcDJBpoxlcRppkP9tLq1oVyR9AvAfwVuhEQjqx05Vni1V78NI+b2rHKDcoIh4XZ+ornD1r4Fn0/TXr6GPQ+Aqw5
wx5HPDE2FhcYXZaI47pejZcjtDcJvlYYvj9cDDqc6jzx11l4ffMOXVoea6M5RHd9pGj5kcf4LbKFb6MXHQe2kUQs4
mSz0jsm5+qLVCL5kF+Q3QZpjqjTRnOThR/IyPYPKx2C65aB6w8t8xOEMnOQ6j+AW8FP05tROwN87RBOZT6e
gVn94zyqbgqkHUAzsnTjaccIYNQ/whtyOOQnisqh/ZIEnTe3nETh3TmOf8q8PL/eLaFoRWochXrcnorHiWi4Bdrme
E5hqW3LFUnZNRbGQ+8OaU10peZYtBjNFPB5sw3CyW2j2QEo9kL4hFsK1VsHXQcdtAIXji3ZmiTtbWvEuHY+eI
JoHctcGIo4WhAfTv0EmNpMyEqn7CAfviUixJYrStQck+bWVToEyydSdYeLLHcjKn3FmX52WJuRWwrQIO/+4IIvIf
Ht/aC4Wy0giq+kDS4g9N8N2lhRTk2LbAC/xQWGmo6PN+d547bICVL2Tm7OMCC9Z0b96Q7rVFAQBgT1sku9br
ig/7GyQIqKXiPRIrD6fK1VbZ7DRkiqung3q013EJrEVqKS6gmOW5bO57LOguHv4r677Jg+Jt9aYLdg7t8kqADxf4G
LkoBvHLXVOVfFaavwOW67fTups9H55LA6TkEYS1BgJqDlfrDmMKViyzJYOWcg/1PfpsgsdcSvlW9/EKZqzwkcV
Uz9TBVrFa/uyukLog3730ifuohW9DRXQozfQSJHVaQf6n2QNTpBB8QIrRLSKSV88rN6caVBeTURzPKU3j1IdZV
evEO22+FQz7013cHOynO6qXxpuhaOP9eGoAe/3DGRAny6tXqfiyy7MB1IWCNLa4LUL6bW13nIxvpYSrvf36C4d
k2aWKaSubLUyqXzlVXCtsQyRlPrZiGMIfIAKM3WAWkzINpYxIDuG/076GNRMgzOCwp4MPXIVz4oa+Sk4o9Lie
OAhVTTeS0qfNowvyiOdxHhtJb2aBeuq/TS10TuYIIH3Dmd+/8FKyC2NvPXA6Uxk/unzKECZfJnURx62/66lI4ZCO
1I+azATD8vwgNH7fk5D2rjSFTzNuYoWrjJ599k6i59aE0kGW6gyfV2bAAU08F6My7iWOoX6zDdjNNudf4IPFljGY
bmwOXpzzx9ArzBasVy3BjUXisaDdA6tj3sz9meLCR7jqlRqj9Oqssc91vXOM9IktOc6/4qZmiKTZxcnKDwSEeGN
W4oG7k9WwLNYZ9ekUN68aGOsoKKzkcI8t3KrYReoskvt1GpZHI1qz1/8prFUX2Yyex0rs69IO4+GfYe6aNLdskU
WuHNOOvpNpk6+ha+vqfIXzNqPrFWUlluDL1cNFe8+7zzqErbsm7o/VoYnisIIHIwOcfHmvHqOjvW6MU6In4tBLgv
wPs8NrsZeCn96rAuwrdSw4M0s0ohmNB/UdXJwmymP8XNXB0XzOLSFgltJebMIPU9vjozip65tHJv4ajn/hbsga9
HN4ENmFhYZEA4/AUTKXCrNloy1pptrR2jFYh+s/PHGDAoMhUuHDnBOp+uznw4dj3y8ya3i+OrPhhPYc2ZZKOs
8gWdlOR8xKWgxMTLPHIEdA+ev31YESzKx81yXt4TnN5xHOKlazIVpG0LXA1Wn3DCEAPou5kDnGJEGRe9Mq
GotvRBBI0djEAzdhhYH54k1p9ojmP7xbyGxYJBUKk8VAXdZ/vHZx1bWTjlaN3RQYxuAsTz1mUu5Fb54WppLRN
SBS15WKT8gWKZ6zFWFb04JgRtjEo1q5GRideObSHn+HBcRtLtGYJ4lUTICVtsiOjl/OHyy+0T32y9AZTkUVQIC
Olbu48Vy1LmWWjv1MRWK0aeKb7FSjPKltX/JQHWenIZsO5Dy3QD8wOcYrbWMHH0crC589QSdqW6wAB5Bfh
D9cMhqGAGL2xi9vqdVSetjvHiGcSF0l+d+78yx9IR4i18saXrMbIgq58+kp8ww+OecTRj1l08HTAF4tFhUlkoCzZyX/
8hqdYr7DGzaoDhGftQ3Nyg4aK8pyt3YDD2JNoL/lcjQAusuhhnYM7Ooi7rxshU4fg1w31INKNODUmQmjh2+C7Bg
LIUyzrl9EFQTAvBrKRNgAJPUuCqkya0JZMiEWP0flVqYpviT5jHuNEhrvDc2D0vP4VMCFvLI5VWjSS3gUyxR8Hi
Gl77VKPfi+IK8DRVid5TA+WFUazEwgaKqBdwKHXhVdNncsrfgsxSVc4tkZOoB577VAuex2X+7yibfjeyGjMNobx3
bB5Phh9y/lf621q9sgmHMv2xp+q8kvVJ5MnLPcObJpBYHHDnzcLI0uK0Tf5V+GUSg/wEla+0FoSkF1OWe91BG
OGWcLZwY5Il9RSoUg965ZImVc

4.- Descifrado de respuesta de VCE

Para descifrar la información que está encriptada de manera simétrica, será necesario contar con los valores que se generaron de manera inicial en el cifrado AES (**viHex::saltHex::passprhase**), estos valores se sugieren que se queden a resguardo por parte del comercio, dado que son la llave que utilizará para descifrar la información que la VCE retorne:

Código:

Se instancia la clase para decodificar:

```
CifradoAES helper = CifradoAES.getInstance();
```

```
public static com.cifrado.utils.CifradoAES getInstance() {
  if (instance == null) {
    instance = new com.cifrado.utils.CifradoAES(128, 1000);
  }
  return instance;
}

private CifradoAES(int keySize, int iterationCount) {
  this.keySize = keySize;
  this.iterationCount = iterationCount;
  try {
    this.cipher = Cipher.getInstance("AES/CTR/NoPadding");
  }
  catch (NoSuchAlgorithmException|javax.crypto.NoSuchPaddingException e) {
    throw fail(e);
  }
}
```

Se invoca el método para descifrar:

```
String descifradoAES = helper.decrypt(request.getSalt(), request.getVi(), request.getPassphrase(), request.getCypherData());
```

```
public String decrypt(String salt, String iv, String passphrase, String ciphertext) {
  try {
    SecretKey key = generateKey(salt, passphrase);
    byte[] decrypted = doFinal(2, key, iv, base64(ciphertext));
    return new String(decrypted, StandardCharsets.UTF_8);
  } catch (Exception e) {
    return null;
  }
}

private byte[] doFinal(int encryptMode, SecretKey key, String iv, byte[] bytes) {
  try {
    this.cipher.init(encryptMode, key, new IvParameterSpec(hex(iv)));
    return this.cipher.doFinal(bytes);
  }
  catch (InvalidKeyException|java.security.InvalidAlgorithmParameterException|javax.crypto.IllegalBlockSizeException|javax.crypto.BadPaddingException e) {
    return null;
  }
}

private SecretKey generateKey(String salt, String passphrase) {
  try {
    SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1");
    KeySpec spec = new PBEKeySpec(passphrase.toCharArray(), hex(salt), this.iterationCount, this.keySize);
    SecretKey key = new SecretKeySpec(factory.generateSecret(spec).getEncoded(), "AES");

    return key;
  }
  catch (NoSuchAlgorithmException|java.security.spec.InvalidKeySpecException e) {
    return null;
  }
}
```

```
public static byte[] hex(String str) {
  try {
    return Hex.decodeHex(str.toCharArray());
  }
  catch (DecoderException e) {
    throw new IllegalStateException(e);
  }
}
```

salt = Elemento **saltHex** generado al momento de realizar el cifrado simétrico en AES.

Ej

3076356C346F7366387675337A316171

iv = Elemento **ivHex** generado al momento de realizar el cifrado simétrico en AES.

Ej

7434316130317A753068723272746569

phassphrase = Elemento **passPhrase** generado al momento de realizar el cifrado simétrico en AES.

Ej.

g-T=Kq81GSgxRhc%

chypertext = Valor del elemento **data** del JSON de respuesta de VCE.

Ej.

{
"**data**":
"XxsiFPMZjPSOjJyanlLHARoa/+lIdegR3lUczOcD4qGa4a4InjrkeL+QY6PpGpJwDoimBOgbB/0E03YQBxNKMIR
X8fZvKxszHcsZ16IgyB/gZHhv+/sguHFMYC0Ccu5hHQejZzX0/5OmXeGkMm6XhlxLKlOhbzcNCaEOQru23snBrIf
voJN8lWC+q4FlQKIGS3TiFLGQGSBahB3ePnysGAiGD4eWrvtRmUM8NdiWJ6AEWKYFtnhAshTc3NprUIjA7IGr
qrHaOtQUw/Zrz7s5ng/Fjj9CI9bE+UyPpjGzPwGtRkkg4G1jGZWPuyJw7yH5tb/q7MboZX8zb5nbVuikDyDhQw==",
"status3D": "200",
"eci": "05",
"decision": "REVIEW",
"reasonCode": 480,
"bnteCode": "00",
"bnteText": "Transaccion Exitosa",
"id": 2,
"message": "operationSuccessfully",
"numeroControl": "PBA22042709695XXFA36H01"
}

Resultado:

{"idAfiliacion":7000002,"referencia":248233587990,"numeroControl":"PBA22042709695XXFA36H01","fechaReqCte":"2022-04-28T13:41:18.558","fechaRspCte":"2022-04-28T13:41:18.684","resultadoPayw":"A","codigoAut":"214414","texto":"Aprobada por Payworks (modo prueba)"}

Formateado como JSON:

{
"idAfiliacion": 7000002,
"referencia": 248233587990,
"numeroControl": "PBA22042709695XXFA36H01",
"fechaReqCte": "2022-04-28T13:41:18.558",
"fechaRspCte": "2022-04-28T13:41:18.684",
"resultadoPayw": "A",
"codigoAut": "214414",
"texto": "Aprobada por Payworks (modo prueba)"
}